

class_Imbalance_B49_AIML_Lab

March 30, 2026

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,
    ↳ ConfusionMatrixDisplay
from sklearn.utils import resample

from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
```

```
[2]: def evaluate_model(y_test, y_pred, title="Model Evaluation"):

    print(f"\n{'='*55}")
    print(f"  {title}")
    print('='*55)
    print(classification_report(y_test, y_pred, target_names=["Class 0",
    ↳ (Majority)", "Class 1 (Minority)"]))

    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    ↳ display_labels=["Majority (0)", "Minority (1)"])
    fig, ax = plt.subplots(figsize=(5, 4))
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(title, fontsize=12, fontweight='bold')
    plt.tight_layout()
    plt.show()
```

```
[3]: # STEP 1: CREATE A SYNTHETIC IMBALANCED DATASET
# We use make_classification() to create a controlled imbalanced dataset.
# weights=[0.90, 0.10] means 90% samples belong to class 0 and only 10% to
    ↳ class 1.
```

```

X, y = make_classification(
    n_samples=2000,
    n_features=10,
    weights=[0.90, 0.10],    # 90% majority, 10% minority
    flip_y=0.01,
    random_state=42
)

# Convert to pandas DataFrame for easy handling
df = pd.DataFrame(X, columns=[f"feature_{i+1}" for i in range(X.shape[1])])
df['target'] = y

print("\nDataset Shape:", df.shape)
print("\nClass Distribution (raw counts):")
print(df['target'].value_counts())
print("\nClass Distribution (percentages):")
print(df['target'].value_counts(normalize=True).map(lambda x: f"{x*100:.1f}%"))

# visualize the imbalance
fig, ax = plt.subplots(figsize=(5, 4))
df['target'].value_counts().plot(kind='bar', color=['steelblue', 'tomato'],
    edgecolor='black', ax=ax)
ax.set_title("Original Class Distribution", fontweight='bold')
ax.set_xlabel("Class")
ax.set_ylabel("Count")
ax.set_xticklabels(["Majority (0)", "Minority (1)"], rotation=0)
plt.tight_layout()
plt.show()

```

Dataset Shape: (2000, 11)

Class Distribution (raw counts):

target

0 1795

1 205

Name: count, dtype: int64

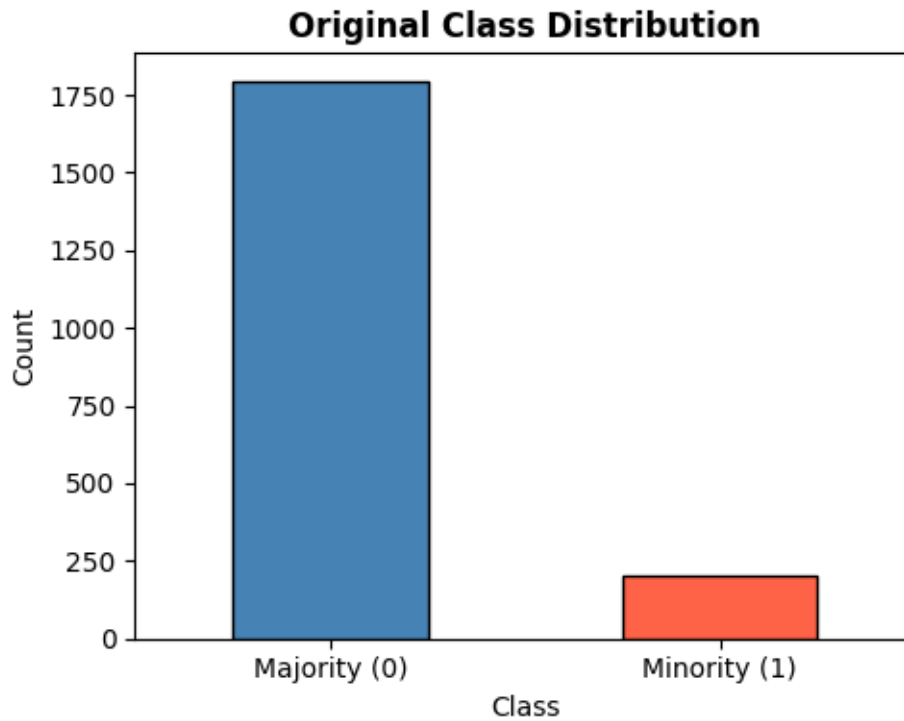
Class Distribution (percentages):

target

0 89.8%

1 10.2%

Name: proportion, dtype: str



```
[4]: # STEP 2: BASELINE MODEL (No Handling)
# First we train a model WITHOUT any imbalance handling.
# This acts as our baseline - notice how the model performs poorly on class 1
# (minority) despite having high overall accuracy. This is the core problem.

X_train, X_test, y_train, y_test = train_test_split(
    df.drop('target', axis=1), df['target'],
    test_size=0.2, random_state=42, stratify=df['target']
)
# stratify ensures that train and test sets have the same class proportions as
# the original dataset.

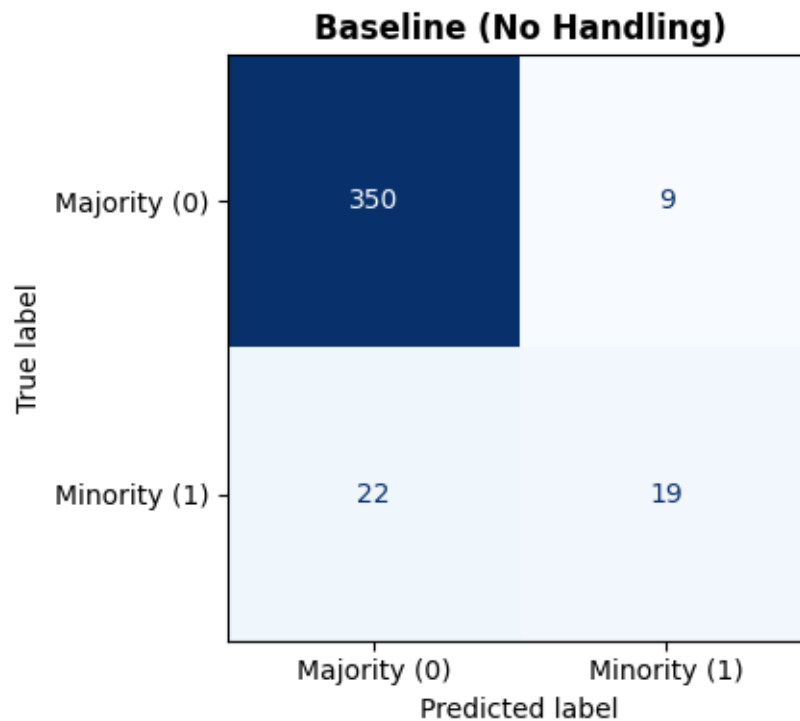
baseline_model = LogisticRegression(random_state=42, max_iter=1000)
baseline_model.fit(X_train, y_train)
y_pred_baseline = baseline_model.predict(X_test)

evaluate_model(y_test, y_pred_baseline, title="Baseline (No Handling)")
```

=====

Baseline (No Handling)

	precision	recall	f1-score	support
Class 0 (Majority)	0.94	0.97	0.96	359
Class 1 (Minority)	0.68	0.46	0.55	41
accuracy			0.92	400
macro avg	0.81	0.72	0.75	400
weighted avg	0.91	0.92	0.92	400



```
[5]: # STEP 3: OVERSAMPLING (Random Oversampling)

# Oversampling duplicates existing minority class samples to balance the
# dataset.
# It does NOT create new information - just repeats existing minority samples.
# RISK: Can lead to overfitting since the same samples are repeated multiple
# times.

# Separate majority and minority classes
X_train_full = pd.concat([X_train, y_train], axis=1)
majority_class = X_train_full[X_train_full['target'] == 0]
minority_class = X_train_full[X_train_full['target'] == 1]
```

```

print(f"\nBefore Oversampling → Majority: {len(majority_class)}, Minority: {len(minority_class)}")

# Upsample minority class to match majority count
minority_upsampled = resample(
    minority_class,
    replace=True, # Sample WITH replacement (allows duplicates)
    n_samples=len(majority_class), # Match majority class size
    random_state=42
)

# Combine back into one balanced dataset
df_oversampled = pd.concat([majority_class, minority_upsampled])
print(f"After Oversampling → {df_oversampled['target'].value_counts().to_dict()}")

X_over = df_oversampled.drop('target', axis=1)
y_over = df_oversampled['target']

os_model = LogisticRegression(random_state=42, max_iter=1000)
os_model.fit(X_over, y_over)
y_pred_os = os_model.predict(X_test)

evaluate_model(y_test, y_pred_os, title="After Random Oversampling")

```

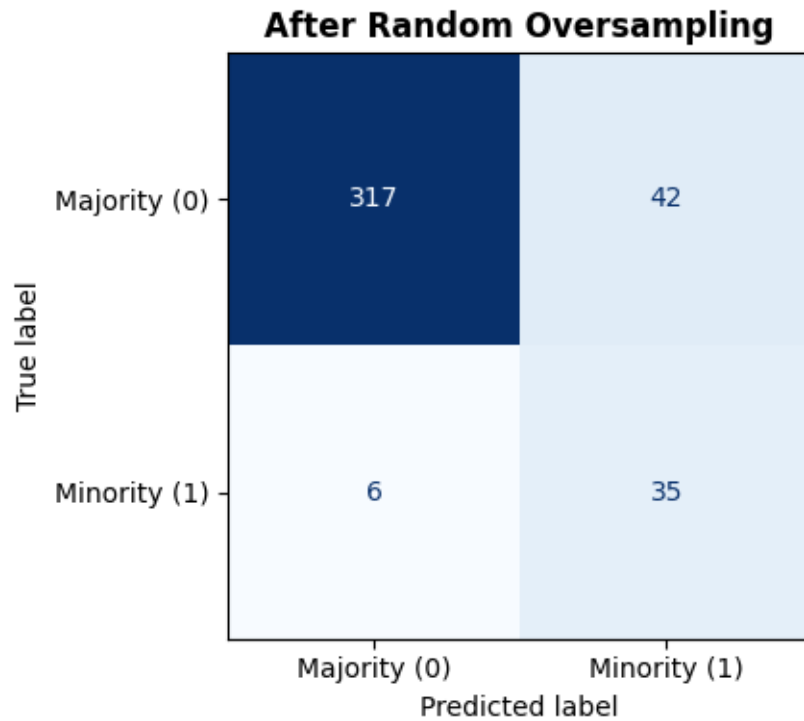
Before Oversampling → Majority: 1436, Minority: 164
 After Oversampling → {0: 1436, 1: 1436}

```

=====
After Random Oversampling
=====

```

	precision	recall	f1-score	support
Class 0 (Majority)	0.98	0.88	0.93	359
Class 1 (Minority)	0.45	0.85	0.59	41
accuracy			0.88	400
macro avg	0.72	0.87	0.76	400
weighted avg	0.93	0.88	0.90	400



```
[7]: # STEP 4: UNDERSAMPLING (Random Undersampling)

# Undersampling reduces the majority class to match the minority class count.
# ADVANTAGE: Faster training, less data needed.
# RISK: Loses potentially valuable information from the majority class.

print(f"\nBefore Undersampling → Majority: {len(majority_class)}, Minority: {len(minority_class)}")

# Downsample majority class to match minority count
majority_downsampled = resample(
    majority_class,
    replace=False,          # Sample WITHOUT replacement (no duplicates)
    n_samples=len(minority_class), # Match minority class size
    random_state=42
)

df_undersampled = pd.concat([majority_downsampled, minority_class])
print(f"After Undersampling → {df_undersampled['target'].value_counts().to_dict()}")

X_under = df_undersampled.drop('target', axis=1)
y_under = df_undersampled['target']
```

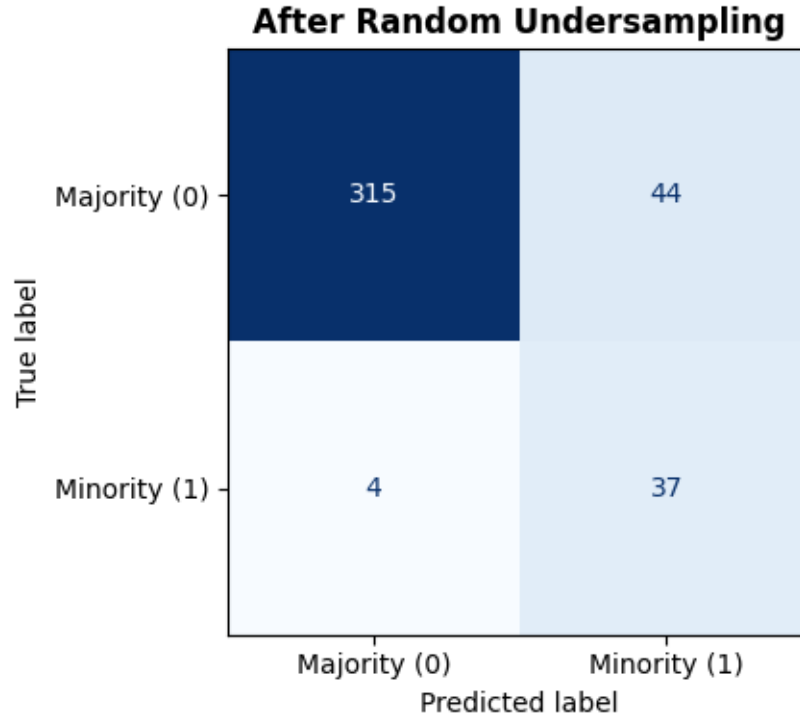
```
us_model = LogisticRegression(random_state=42, max_iter=1000)
us_model.fit(X_under, y_under)
y_pred_us = us_model.predict(X_test)

evaluate_model(y_test, y_pred_us, title="After Random Undersampling")
```

Before Undersampling → Majority: 1436, Minority: 164

After Undersampling → {0: 164, 1: 164}

After Random Undersampling				
	precision	recall	f1-score	support
Class 0 (Majority)	0.99	0.88	0.93	359
Class 1 (Minority)	0.46	0.90	0.61	41
accuracy			0.88	400
macro avg	0.72	0.89	0.77	400
weighted avg	0.93	0.88	0.90	400



```
[8]: # STEP 5: SMOTE (Synthetic Minority Over-sampling Technique)

# SMOTE is smarter than random oversampling. Instead of just duplicating
↳ samples,
# it CREATES NEW SYNTHETIC samples by interpolating between existing minority
↳ samples.
# HOW: It picks a minority sample, finds its K nearest neighbors, and creates
# a new point along the line between them.
# ADVANTAGE: More generalizable than simple duplication - model sees diverse
↳ examples.
# LIBRARY REQUIRED: imbalanced-learn (pip install imbalanced-learn)

smote = SMOTE(random_state=42)
X_smote, y_smote = smote.fit_resample(X_train, y_train)

print(f"\nBefore SMOTE → {pd.Series(y_train).value_counts().to_dict()}")
print(f"After SMOTE → {pd.Series(y_smote).value_counts().to_dict()}")

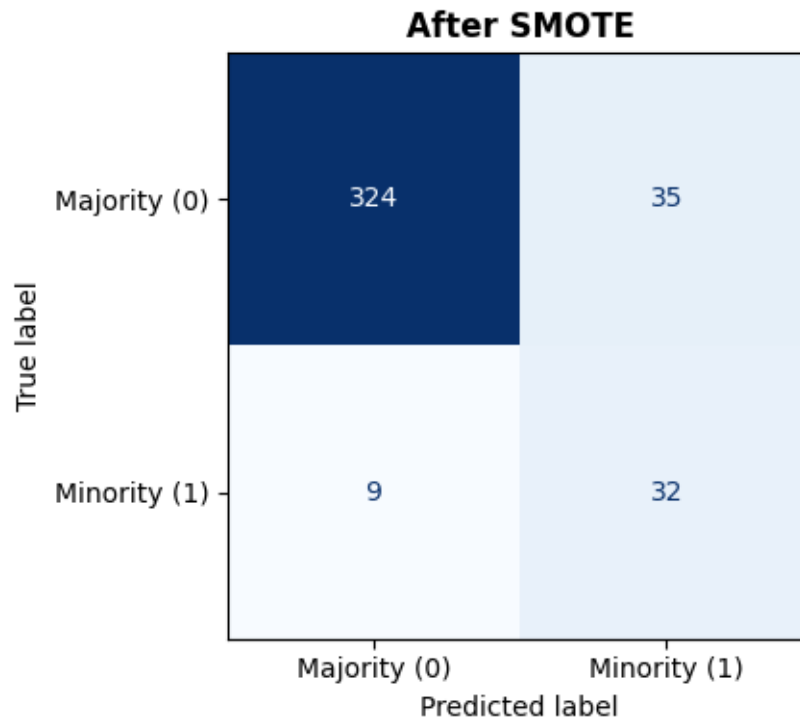
smote_model = LogisticRegression(random_state=42, max_iter=1000)
smote_model.fit(X_smote, y_smote)
y_pred_smote = smote_model.predict(X_test)

evaluate_model(y_test, y_pred_smote, title="After SMOTE")
```

Before SMOTE → {0: 1436, 1: 164}
After SMOTE → {0: 1436, 1: 1436}

```
=====
After SMOTE
=====
```

	precision	recall	f1-score	support
Class 0 (Majority)	0.97	0.90	0.94	359
Class 1 (Minority)	0.48	0.78	0.59	41
accuracy			0.89	400
macro avg	0.73	0.84	0.76	400
weighted avg	0.92	0.89	0.90	400



```
[ ]: # STEP 6: CLASS WEIGHTS (Cost-Sensitive Learning)

# Instead of changing the data, we tell the model to PENALIZE mistakes on the
# minority class MORE than on the majority class.
# class_weight='balanced' automatically calculates weights inversely
  ↳proportional
# to class frequencies: rare class → higher weight → bigger penalty for wrong
  ↳predictions.
# ADVANTAGE: No data modification needed - works directly within the algorithm.

from sklearn.linear_model import LogisticRegression
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

# Train model
cw_model = LogisticRegression(class_weight='balanced', random_state=42,
  ↳max_iter=1000)
cw_model.fit(X_train, y_train)
y_pred_cw = cw_model.predict(X_test)

# Correct way to compute and display class weights
classes = np.unique(y_train)
```

```

weights = compute_class_weight(class_weight='balanced', classes=classes,
↪y=y_train)

print("\nAuto-assigned class weights:",
      dict(zip(classes, np.round(weights, 2))))
print("(Higher weight = more penalty for misclassifying that class)")

# Evaluate model
evaluate_model(y_test, y_pred_cw, title="After Class Weighting")

```

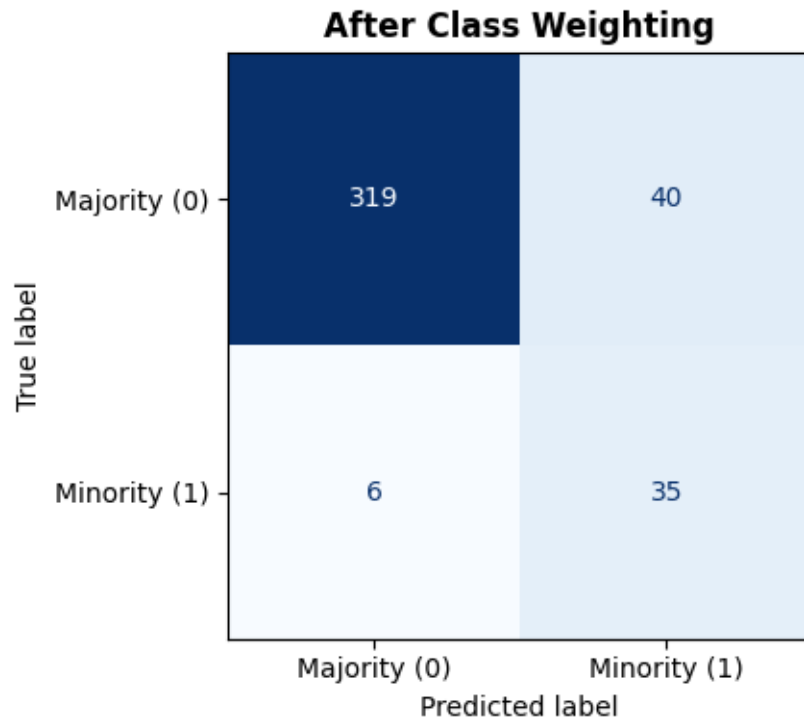
Auto-assigned class weights: {np.int64(0): np.float64(0.56), np.int64(1):
np.float64(4.88)}
(Higher weight = more penalty for misclassifying that class)

```

=====
After Class Weighting
=====

```

	precision	recall	f1-score	support
Class 0 (Majority)	0.98	0.89	0.93	359
Class 1 (Minority)	0.47	0.85	0.60	41
accuracy			0.89	400
macro avg	0.72	0.87	0.77	400
weighted avg	0.93	0.89	0.90	400



```
[14]: # STEP 7: THRESHOLD TUNING

# By default, classifiers use 0.5 as the decision threshold.
# i.e., if predicted probability > 0.5 → predict class 1 (minority).
# If we LOWER the threshold (e.g., to 0.3), the model becomes more aggressive
# about predicting the minority class → better recall but may lower precision.
# TRADEOFF: There is always a recall vs precision tradeoff. Threshold tuning
# lets us manually control this balance based on business needs.

# Get predicted probabilities (not just class labels)
y_prob = baseline_model.predict_proba(X_test)[: , 1] # Probability of being
↳ Class 1

threshold = 0.3 # Lower than default 0.5 → more sensitive to minority class
y_pred_thresh = (y_prob >= threshold).astype(int)

print(f"\nUsing threshold = {threshold} (default is 0.5)")
print("Lower threshold → model flags more cases as minority class")

evaluate_model(y_test, y_pred_thresh, title=f"Threshold Tuning_
↳ (threshold={threshold})")
```

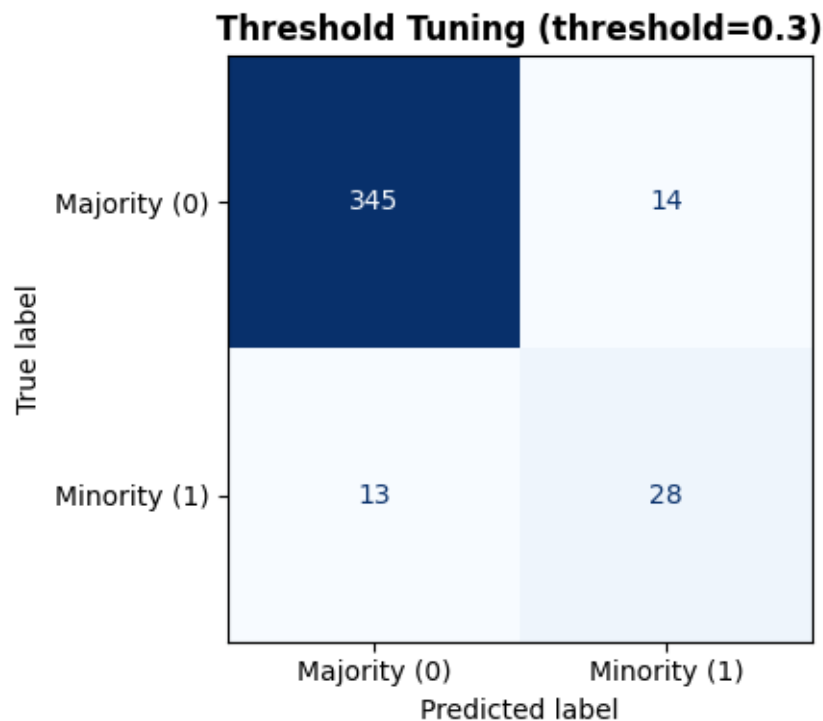
Using threshold = 0.3 (default is 0.5)
 Lower threshold → model flags more cases as minority class

```
=====
```

Threshold Tuning (threshold=0.3)

```
=====
```

	precision	recall	f1-score	support
Class 0 (Majority)	0.96	0.96	0.96	359
Class 1 (Minority)	0.67	0.68	0.67	41
accuracy			0.93	400
macro avg	0.82	0.82	0.82	400
weighted avg	0.93	0.93	0.93	400



```
[15]: # STEP 8: COMPARISON SUMMARY

# Let's now compare all techniques side-by-side.
# We focus on F1-score for the minority class (Class 1) as our key metric,
# since it balances both precision and recall - important for imbalanced
# problems.
```

```

from sklearn.metrics import f1_score, recall_score, precision_score

results = {
    "Baseline (No Handling)": y_pred_baseline,
    "Random Oversampling": y_pred_os,
    "Random Undersampling": y_pred_us,
    "SMOTE": y_pred_smote,
    "Class Weights": y_pred_cw,
    f"Threshold Tuning (t={threshold})": y_pred_thresh,
}

summary = []
for name, preds in results.items():
    summary.append({
        "Technique": name,
        "Precision (Class 1)": round(precision_score(y_test, preds,
↪pos_label=1), 3),
        "Recall (Class 1)": round(recall_score(y_test, preds, pos_label=1),
↪3),
        "F1-Score (Class 1)": round(f1_score(y_test, preds, pos_label=1), 3),
    })

summary_df = pd.DataFrame(summary).set_index("Technique")
print("\n", summary_df.to_string())

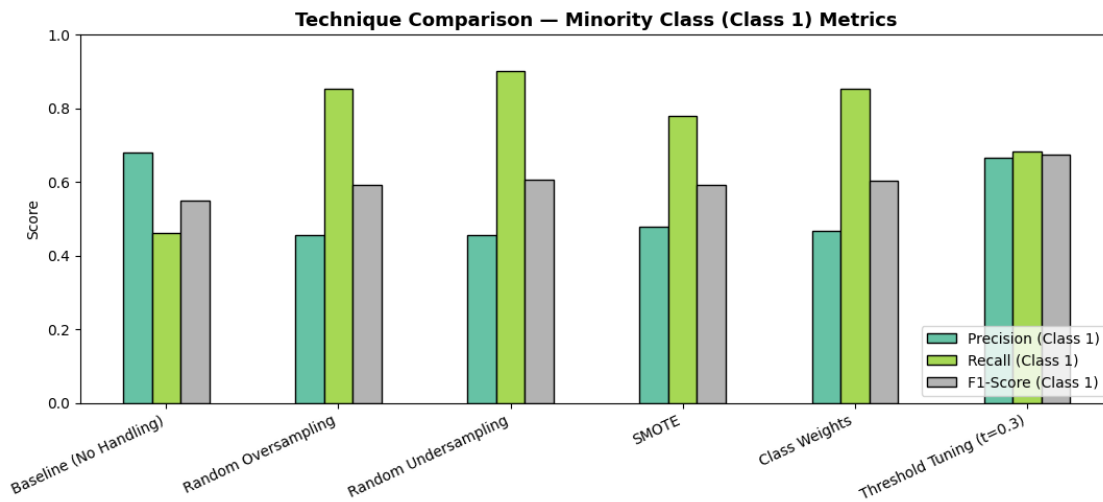
# Visual bar chart comparison
summary_df.plot(kind='bar', figsize=(11, 5), colormap='Set2', edgecolor='black')
plt.title("Technique Comparison - Minority Class (Class 1) Metrics",
↪fontweight='bold', fontsize=13)
plt.ylabel("Score")
plt.xlabel("")
plt.xticks(rotation=25, ha='right')
plt.ylim(0, 1)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

print("KEY TAKEAWAY: No single technique is universally best.")
print("Choose based on your dataset size, domain, and the cost of false
↪negatives vs false positives.")

```

	Precision (Class 1)	Recall (Class 1)	F1-Score
(Class 1)			
Technique			
Baseline (No Handling)	0.679	0.463	
0.551			

Random Oversampling	0.455	0.854
0.593		
Random Undersampling	0.457	0.902
0.607		
SMOTE	0.478	0.780
0.593		
Class Weights	0.467	0.854
0.603		
Threshold Tuning ($t=0.3$)	0.667	0.683
0.675		



KEY TAKEAWAY: No single technique is universally best.
Choose based on your dataset size, domain, and the cost of false negatives vs false positives.

[]: